

1-Question: What is the shortcut to comment and uncomment a selected block of code in Visual Studio?

A:

Ctrl + K, Ctrl + U

2-Question: Explain the difference between a runtime error and a logical error with examples.

A:

Runtime Error: This error appears when something like dividing 4 by zero happens. We use defensive programming or protective code to handle it.

Logical Error: Everything looks fine and runs, but during debugging, we monitor the code line by line to find the problem.

3-Question: Why is it important to follow naming conventions such as PascalCase in C#?

A:

Improves Readability It makes the code much easier to read and understand and Team Collaboration and Maintainability: It helps other developers who will work on the code after us to understand it quickly and continue working on it without confusion.

4-Question: Explain the difference between value types and reference types in terms of memory allocation.

A:

Value Types: They store the actual value directly in the variable, and we can predict their size in memory (typically stored on the Stack).

Reference Types: They store a reference (memory address) that points to the value, not the value itself, and we cannot easily predict their size (typically stored on the Heap).

5-Question: What will be the output of the following code? Explain why:

```
int a = 2, b = 7;
```

```
Console.WriteLine(a % b);
```

A:

The output is 2 because 2 cannot be divided by 7 (it contains zero 7s), so the remainder is the number 2 itself.

6-Question: How does the && (logical AND) operator differ from the & (bitwise AND) operator?

A

Logical AND: Works with boolean values (true/false). It uses short-circuiting, meaning if the first condition is false, it doesn't check the second one.

Bitwise AND: Works at the binary level, comparing bits (0s and 1s) of numbers. If used with boolean values, it checks both sides completely without short-circuiting.

7-Question: Why is explicit casting required when converting a double to an int?

A:

Explicit casting is required because a double can hold larger values and fractional numbers that an int cannot. We must explicitly cast to acknowledge the potential for data loss (losing the decimal part) or overflow, which can be handled using a checked block.

8-Question: What exception might occur if the input is invalid and how can you handle it.

A:

Exception: If we use `int.Parse` and the user enters letters or symbols instead of numbers, the program will crash and throw a `FormatException`. We can handle it by using `int.TryParse` with an if condition (Defensive Programming). This safely checks if the input is valid and converts it without crashing the program.

9-Question: Given the code below, what is the value of x after execution? Explain why

```
int x = 5;  
int y = ++x + x++;
```

A:

- 1) ++x increments x from 5 to 6 and uses 6.
- 2) x++ uses the current value of x (which is 6) and then increments x to 7.
- 3) $y = 6 + 6 = 12$, and the final value of x is 7.

10- LinkedIn article about variables allocation in stack and heap for both value and ref types .

Today I study something very important in C# memory, I want to share what I learn about Stack and Heap.

In C#, we have two types of variables:

1. Value Types (like int, bool).
2. Reference Types (like string, arrays).

Where do they live in memory? Value Types go to the "Stack"., The Stack is very fast memory and saves the real value directly, Reference Types go to the "Heap". The Heap is a big memory for big things, The Stack only saves the address (reference) that points to the Heap.

11- what's the difference between compiled and interpreted languages and in this way what about Csharp?

A:

Compiled languages: They translate all the code to machine code before the program starts. They are very fast.

Interpreted languages: They translate the code line by line while the program is running. They are slower but easy to test.

C# Language: C# is a mix of both First, it compiles the code into an Intermediate Language (IL). Then, when we run the program, the JIT compiler translates it to machine code.

12- Compare between implicit, explicit, Convert and parse casting.

A:

Implicit Casting: The compiler does it automatically. It is very safe. Like changing a small type to a big type (int to double).

Explicit Casting: We write it manually using (). It is not safe because we can lose data (like decimal numbers). Like changing a big type to a small type (double to int).

Parse Casting: We use it to convert a string to a number (like int.Parse). It only works with strings.

Convert Class: We use it to change any type to another type (like Convert.ToInt32). It is good because if the value is null, it returns 0 and does not crash the program.

